

Monotone Classification with Relative Approximations

Lecture Notes for SC2320 Data Analytics and Mining

April 7, 2026

Instructions to readers

The flow of this set of notes is in tandem to the flow of the original paper. It will also go in the same flow as the presentation slides. Concepts/Material/Information that is auxiliary to the core ideas of this paper will be outlined at the end of each section.

Acronym Reference

Acronym	Full Form
ER	Entity Resoluton
EM	Entity Matching
ACR3	Acronym Three

1 Background and Motivation

1.1 The Challenge of Entity Matching

Definition 1.1 (Entity Matching). Given two sets of entities E_1 and E_2 , the task is to determine, for every pair $(e_1, e_2) \in E_1 \times E_2$, whether e_1 and e_2 refer to the same real-world entity. It is a binary classification task that determines whether a candidate pair is a match (+1) or non-match (-1) [2].

To motivate the problem, consider the following scenario where we have two sets of **entities** that we refer to here as product listings - E_1 and E_2 , sourced from two different e-commerce platforms — say Shopee and Lazada respectively. Each listing carries attributes such as **p-name**, **p-description**, **p-year**, and **p-price**. The goal of Entity Matching in this scenario is to decide, for every pair $(e_1, e_2) \in E_1 \times E_2$, if they are referring to the same real-world product.

What makes this non-trivial is that even a genuinely matching pair may disagree on attribute values. For instance, one platform may list a product as “*Sony WH-1000XM5*” while another lists it as “*Sony Noise Cancelling Headphones XM5 Series*”. They are referring to the same product but with different representations. It is therefore insufficient to rely solely on attribute-level comparison to determine a match. To attain maximum accuracy in entity matching, it falls upon a human expert to manually inspect each pair and make a verdict. However, with thousands of listings on each platform, the number of pairs in $E_1 \times E_2$ grows quadratically - making manual inspection labor intensive and expensive. This motivates the need for a more systematic and efficient approach to Entity Matching.

1.2 The Entity Resolution (ER) Pipeline

To address the Entity Matching Problem, we transform it into a classification problem. This is done by passing datasets through an ER Pipeline consisting of three key steps: **Blocking**, **Comparison**, and **Matching**. The first two steps serve as preprocessing stages that progressively reduce and structure the problem, while the third step is what Entity Matching entails and is the primary focus of the paper. We will go over the first two steps in brief before diving into greater detail on the third.

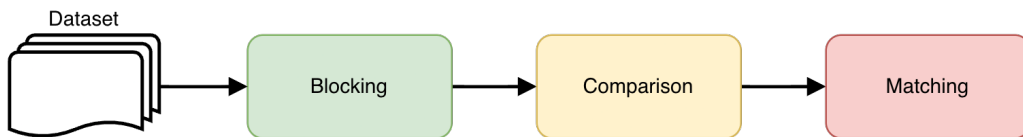


Figure 1: The ER Pipeline.

1.2.1 Blocking

As mentioned earlier in 1.1, the two entity sets E_1 and E_2 can grow quadratically, making it computationally infeasible to compare every pair. The goal of blocking is to narrow the search space down to a smaller candidate subset $S \subseteq E_1 \times E_2$, retaining only pairs that are plausible matches based on an application-dependent heuristic.

Going back to the Shopee-Lazada example, one may restrict S to only those pairs (e_1, e_2) where **p-year** agrees across both listings. Note that blocking is optional — if skipped, $S = E_1 \times E_2$.

Remark 1.1 (Application-Dependent Heuristic). The criterion used for blocking is **application-dependent**; there is no universal rule for what constitutes an “obvious non-match”.

For example:

- **E-commerce:** A mismatched **p-year** is a reliable signal to discard a pair.
- **Hospital records:** Mismatched blood type might serve that role.
- **Legal documents:** Neither attribute would reliably apply.

The heuristic is therefore dictated by which attributes are reliable and meaningful in the specific domain.

A notable example of a more general, domain-agnostic blocking heuristic is **Locality Sensitive Hashing (LSH)**. It groups similar pairs into candidate buckets based on hash collisions, this was covered in Chapter 3 of this course.

1.2.2 Comparison

For each remaining pair $(e_1, e_2) \in S$, the comparison step quantifies their similarity across d attributes using a set of similarity functions $sim_1, sim_2, \dots, sim_d$. The i -th coordinate of the resulting point p_{e_1, e_2} equals $sim_i(e_1, e_2)$, where a higher value indicates greater similarity on that attribute. The choice of similarity function depends on the attribute type:

- For numerical attributes such as **p-price**, one may use $-|e_1.\text{price} - e_2.\text{price}|$, where the negation ensures consistency with the convention that higher values reflect greater similarity.
- For text attributes such as **p-name** and **p-description**, functions such as edit distance, Jaccard similarity, or cosine similarity may be employed.

Example 1.1. Consider an entity pair $(e_1, e_2) \in S$. e_1 is a Shopee listing and e_2 is a Lazada listing for what may be the same product. Suppose each entity has the attributes **p-name**, **p-description**, **p-year**, and **p-price**. We pass each attribute pair into a pre-defined similarity function as follows:

- **p-name:** a text attribute — use Jaccard similarity. E.g., “*Sony WH-1000XM5*” vs “*Sony Noise Cancelling XM5*” $\rightarrow sim_1(e_1, e_2) = 0.8$
- **p-description:** a long text attribute — use cosine similarity. $\rightarrow sim_2(e_1, e_2) = 0.65$
- **p-year:** a numerical attribute — use exact match, returning 1 if equal and 0 otherwise. $\rightarrow sim_3(e_1, e_2) = 1$
- **p-price:** a numerical attribute — use $-|e_1.\text{price} - e_2.\text{price}|$. E.g., \$350 vs \$345 $\rightarrow sim_4(e_1, e_2) = -5$

The resulting point representing this pair in $\mathbb{R}^4$ is:

$$p_{e_1, e_2} = (0.8, 0.65, 1, -5)$$

This will be done for every pair in S .

This process yields a d -dimensional set of points $P = \{p_{e_1, e_2} \mid (e_1, e_2) \in S\}$ which is handed over to the Matching step.

1.2.3 Matching

The entity matching objective now becomes a binary classification task of assigning each point $p_{e_1, e_2} \in P$ to its correct ground truth label, which is $+1$ for a matching pair and -1 otherwise. While the ultimate approach to this would be with human judgement, this paper introduces **Monotone Classification** as a way to automate the process while ensuring a high degree of labelling accuracy.

1.3 Monotone Classification

Instead of relying on human judgement, we want to find a classifier to correctly label matching and non-matching entities.

The input is the set P of n points in $\mathbb{R}^d$ for $d \geq 1$ that we obtained from the Comparison step. Each point $p \in P$ holds a label from $\{-1, 1\}$ which is represented as $label(p)$.

We first define a classifier as a function:

$$h : \mathbb{R}^d \rightarrow \{-1, +1\}$$

where $h(p) = label(p)$ is a correct classification made by h and will be a misclassification otherwise.

The question is, what would make a good classifier in this context?

1.3.1 The Monotonicity Constraint

To answer this, we first introduce the notion of **dominance**. We say that a point $p \in \mathbb{R}^d$ *dominates* another point $q \in \mathbb{R}^d$, written $p \succ q$, if:

$$p[i] \geq q[i] \quad \text{for all } i \in [d]$$

That is, p scores at least as high as q on every attribute. In the context of entity matching, this means that the pair represented by p is at least as similar as the pair represented by q on every feature.

This supports the **Monotonicity Constraint**. A classifier h is said to be **monotone** if:

$$p \succ q \implies h(p) \geq h(q)$$

Intuitively, if a pair p is at least as similar as q on every attribute, it would be contradictory for h to label p as a non-match (-1) while labeling q as a match ($+1$). A more similar pair should receive at least as positive a label. We denote the set of all monotone classifiers as $\mathbb{H}_{mon}$. So for entity matching, the key criteria of our classifier is that it must be **monotone**.

1.3.2 The Optimal Monotone Error

Given a classifier h , its **error** on P is defined as the number of points it misclassifies:

$$err_P(h) = \sum_{p \in P} \mathbf{1}_{h(p) \neq label(p)}$$

That is, the total count of points in P whose assigned label differs from their ground truth. A classifier $h \in \mathbb{H}_{mon}$ is said to be **optimal** on P if it achieves the smallest possible error among all monotone classifiers. We define this minimum achievable error as the **Optimal Monotone Error**:

$$k^* = \min_{h \in \mathbb{H}_{\text{mon}}} \text{err}_P(h)$$

Note that k^* need not be zero — in practice, the labels in P may not perfectly obey monotonicity due to noise in the data, meaning even the best monotone classifier will misclassify some points.

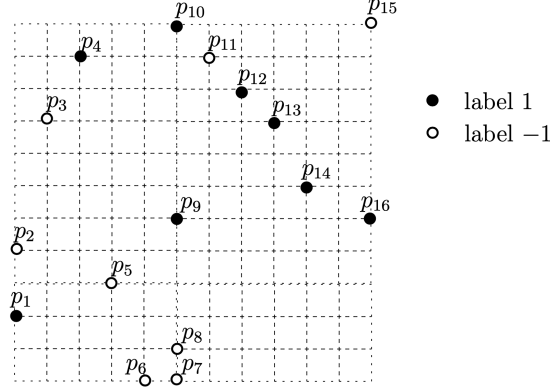


Figure 2: Labelling of input set P by monotone classifier h

Example 1.2. Consider the 2D point set P illustrated in Figure 2 taken from Professor Tao’s original paper [2]. Each point $p_{e_1, e_2} \in P$ represents an entity pair with $d = 2$ similarity features. Black points carry label +1 (match) and white points carry label -1 (non-match).

The monotone classifier h that maps all black points to +1 except p_1 , and all white points to -1 except p_{11} and p_{15} , achieves $\text{err}_P(h) = 3$. No other monotone classifier does better on P , and hence $k^* = 3$. Note that $k^* > 0$ — this is the non-realizable case, as no monotone classifier can perfectly separate all points in P .

We now want an algorithm $\mathcal{A}$ to find a classifier from $\mathbb{H}_{\text{mon}}$ with low error on P . Note that in the beginning, the labels on all points in P are hidden. In order for $\mathcal{A}$ to gain better understanding in forming an optimal classifier, it is allowed to query an *oracle* — a human expert in practice — to reveal the ground truth label of a chosen point $p \in P$. This action is defined as a *probe*. To assess the efficiency of the $\mathcal{A}$, we can measure its cost by the number of *probes* it makes.

To find an optimal monotone classifier on P , one can simply have $\mathcal{A}$ to probe all of P with a cost of n . On the other hand, not probing at all would incur zero cost but the classifier may perform poorly on P . This presents us with the challenge of minimizing the number of probes while still producing a classifier with sufficiently low error.

Formally, given a tolerance parameter $\epsilon \geq 0$, the goal is to determine the minimum number of probes required to guarantee a classifier with error at most $(1 + \epsilon) \cdot k^*$. This is defined formally below.

Problem 1.1 (Monotone Classification with Relative Precision Guarantees). Given a real value $\epsilon \geq 0$, find a monotone classifier $h \in \mathbb{H}_{\text{mon}}$ whose error on P is at most $(1 + \epsilon) \cdot k^*$, while minimizing the number of probes made by $\mathcal{A}$.

Note that when $\epsilon = 0$, the algorithm must find an exactly optimal classifier, which as we will see requires $\Omega(n)$ probes in the worst case — essentially probing all of P . This justifies the introduction of $\epsilon > 0$, where even a small tolerance dramatically reduces the probing cost. The

remainder of this paper studies how efficiently this problem can be solved across the full range of ϵ .

1.4 Active Classification (Supplementary)

To motivate Active Classification, we first revisit a familiar example of **passive classification** — the **Perceptron Algorithm**, covered in SC2300 Machine Learning. Recall that the Perceptron is a mistake-driven algorithm that iterates through all training examples $S_n = \{(x^{(t)}, y^{(t)}), t = 1, \dots, n\}$, where both the points *and* their labels are fully visible from the start. It updates its parameter vector θ whenever a misclassification occurs, converging to a linear classifier that correctly separates all training examples — provided they are linearly separable. This is the **realizable case**.

Remark 1.2 (Realizable Case). A classification problem is **realizable** if there exists a classifier $h \in \mathbb{H}_{mon}$ that correctly classifies all points in P with zero error, i.e., $k^* = 0$. This is analogous to linear separability in the Perceptron setting.

When data is not linearly separable, the Perceptron fails to converge — motivating the use of **hinge loss** to tolerate a margin of error. The same challenge arises here: in practice, the labels in P rarely obey monotonicity perfectly, meaning no monotone classifier can achieve zero error. This is the **non-realizable case**.

Remark 1.3 (Non-Realizable Case). A classification problem is **non-realizable** if no classifier $h \in \mathbb{H}_{mon}$ can achieve zero error on P , i.e., $k^* > 0$. This arises when the labels in P contain noise — points that violate monotonicity regardless of which classifier is chosen. This is analogous to the non-linearly separable case in SC2300.

Remark 1.4. Just as hinge loss introduces a tolerance for misclassified points in the non-separable case, this paper introduces $(1 + \epsilon)k^*$ as a multiplicative tolerance for the non-realizable case — same philosophy, different context.

In the context of Entity Matching, the non-realizable case is the norm. Furthermore, unlike the Perceptron, labels are **not** visible from the start — they are hidden and can only be revealed through probing. This is where **Active Classification** comes in. Devise a learning strategy where the algorithm selectively chooses which points to probe rather than requiring all labels upfront, gathering just enough evidence to construct a classifier with error at most $(1 + \epsilon)k^*$ while minimizing the number of probes. In other words, Active Classification takes into account cost apart from just accuracy.

The contrast with passive classification is summarized below:

	Passive (Perceptron)	Active (This Paper)
Labels	All visible from start	Hidden, revealed via probes
Setting	Realizable ($k^* = 0$)	Non-realizable ($k^* > 0$)
Cost metric	Number of iterations	Number of probes
Goal	Zero training error	$(1 + \epsilon)k^*$ error

Table 1: Passive vs. Active Classification

A key limitation of existing active learning approaches is that they require prior knowledge of k^* — the optimal monotone error — before the algorithm even begins. This is an unrealistic

assumption in practice, since k^* can only be determined after all labels are revealed. This paper proposes algorithms that require no such prior knowledge, working instead by strategically probing points to implicitly converge towards a $(1 + \epsilon)$ -approximate solution — which we examine in the sections that follow.

2 RPE Algorithm

Having formulated the problem in Section 1.1, we now turn to our first algorithm for solving it: the Random Probes with Elimination (RPE) algorithm. This algorithm is the paper’s primary contribution for the realizable case ($k^*=0$) and serves as the foundation for the more general coresset-based approach in Section 3.

2.1 The Algorithm

RPE repeatedly performs two steps:

1. Pick a random point and query its label
2. Use monotonicity to eliminate all points whose labels are now determined

algorithm RPE(P):

1. $Z = \emptyset$
2. **while** $P \neq \emptyset$ **do**
3. probe an element $z \in P$ chosen uniformly at random and add z to Z
4. **if** $\text{label}(z) = 1$ **then remove from** P every $p \in P$ satisfying $p \succeq z$
5. **else remove from** P every $p \in P$ satisfying $z \succeq p$
6. /* note: z is removed by Line 4 or 5 */
6. **return** Z

Remark 2.1 (Intuition). If a point z is positive, then every point above it must also be positive $\rightarrow$ remove them. If z is negative, then every point below it must also be negative $\rightarrow$ remove them. Thus, each probe “wipes out” a whole region of points.

The set Z returned by RPE is the set of all probed points. The classifier produced from Z is:

$$h_{\text{RPE}}(p) = \begin{cases} 1 & \text{if } \exists z \in Z \text{ such that } \text{label}(z) = 1 \text{ and } p \succeq z, \\ -1 & \text{otherwise.} \end{cases}$$

That is, a point p is classified as positive if it lies above any known positive point; otherwise it is classified as negative. Thus, the algorithm only needs to remember: “Where are the confirmed positive points?”

Why is the Classifier always Monotone?

Suppose for contradiction that there exist points $p \succ q$ such that $h_{\text{RPE}}(p) = -1$ and $h_{\text{RPE}}(q) = 1$. Since $h_{\text{RPE}}(q) = 1$, there exists some $z \in Z$ with $\text{label}(z) = 1$ such that $q \succeq z$. But then $p \succ q \succeq z$, so $p \succeq z$ as well. By the definition of h_{RPE} , this implies $h_{\text{RPE}}(p) = 1$, contradicting the assumption that $h_{\text{RPE}}(p) = -1$.

Remark 2.2 (Intuition). If a point q is labelled $+1$, it must dominate some positive probe z in Z . If another point p dominates q , then by transitivity ($p \succeq q \succeq z$), p also dominates that same positive probe z and must therefore also be $+1$.

2.2 Why It Works

The efficiency of RPE stems from the fact that each probe does “double duty”. When RPE probes a point z and learns its label, monotonicity immediately determines the labels of an entire region points that are comparable to z .

- If $\text{label}(z) = 1$: any point $p \succeq z$ is at least as similar as z on every feature. Monotonicity forces $h(p) \geq h(z) = 1$, so all such p must be labelled $+1$ and can be removed from P .
- If $\text{label}(z) = -1$: any point $p \preceq z$ is dominated by z , so monotonicity forces $h(p) \leq h(z) = -1$. All such p must be labelled -1 and are removed.

Only points whose label cannot yet be inferred by monotonicity remain in P after each elimination step. These are the points the algorithm must still resolve, and it will probe them in subsequent iterations.

2.3 A Worked Example: RPE in Action

We trace RPE on the 2D point set P from Figure 2. Recall that black points have label $+1$ and white points have label -1 . The optimal monotone classifier achieves $k^* = 3$ by misclassifying only p_1, p_{11}, p_{15} . The dominance width of this input is $w = 6$.

Example 2.1 (from paper, Example 1.3). Suppose RPE randomly probes p_1 first.

- **Step 1:** Probe p_1 . The oracle reveals $\text{label}(p_1) = 1$. RPE removes every $p \succeq p_1$. Since p_1 sits towards the bottom-left of the figure, it dominates (or equals) almost all other points. The entire P is eliminated except p_6, p_7, p_8 (which lie below or incomparable to p_1).
- **Step 2:** Probe p_8 . The oracle reveals $\text{label}(p_8) = -1$. RPE removes every $p \preceq p_8$, which eliminates p_6, p_7 and p_8 itself. P is now empty. RPE terminates with $Z = \{p_1, p_8\}$.
- **Classifier output:** Since $\text{label}(p_1) = 1$, every point $p \succeq p_1$ is mapped to $+1$ by h_{RPE} . All other points (including p_6, p_7, p_8) are mapped to -1 .
- **Error:** $\text{err}_P(h_{\text{RPE}}) = 5$ (5 white points mapped to $+1$ due to dominating p_1).

This example demonstrates how a single probe of p_1 is enough to determine the labels of 13 out of 16 points. The cost of 2 probes is far below the naïve cost of $n = 16$. Note, however, that the error of 5 exceeds $k^* = 3$ — we will see why this is expected and bounded by the $2k^*$ guarantee.

2.4 Guarantees

The following theorem (proved in full in the paper) captures the performance of RPE:

Theorem 2.1 (RPE Guarantees). For Problem 1.1, the RPE algorithm satisfies:

- **Expected error:** The classifier h_{RPE} has expected error at most $2k^*$:

$$\mathbb{E}[\text{err}_P(h_{\text{RPE}})] \leq 2k^*.$$

- **Expected cost:** RPE probes at most $O(w \cdot \log(n/w))$ elements in expectation, where w is the dominance width of P , $n = |P|$, and $\log(x) = \log_2(1 + x)$.

Two aspects of this theorem are worth emphasizing. First, RPE never needs to compute w — the width is only used in the analysis. Second, when $k^* = 0$ (the realizable case), the expected error guarantee of $2k^* = 0$ means RPE always finds an optimal classifier.

Proof Sketch

1. Error Analysis

Fix any monotone classifier $h \in \mathbb{H}_{\text{mon}}$. Call a point $p \in P$ *h-good* if $h(p) = \text{label}(p)$ and *h-bad* otherwise. The proof proceeds by induction on $|P|$, showing:

Lemma 2.1. The number of *h-good* elements misclassified by h_{RPE} is at most $\text{err}_P(h)$ in expectation.

Setting $h = h^*$ (the optimal classifier), there are exactly k^* h^* -bad elements. The total expected error is thus $\leq k^* + k^* = 2k^*$.

The central tool in the inductive argument is a **pairing between influence sets**. When RPE probes a point z , it may cause other points to be misclassified:

- For an *h-bad* point p , $I_{\text{bad}}(p)$ is the set of *h-good* points that get forced into the wrong label if p is probed first.
- For an *h-good* point q , $I_{\text{good}}(q)$ is the set of *h-bad* points that get forced into the correct label (according to h) if q is probed first.

The key insight is that these influences balance perfectly:

Lemma 2.2. For any *h-bad* $p \in P$ and *h-good* $q \in P$, $q \in I_{\text{bad}}(p) \iff p \in I_{\text{good}}(q)$. Hence, $\sum_{p \in \text{bad}} |I_{\text{bad}}(p)| = \sum_{q \in \text{good}} |I_{\text{good}}(q)|$.

Why does this give the $2k^*$ bound on the total expected error? Because the total “potential for error” (left side) equals the total “potential for correction” (right side), the expected number of *h-good* points misclassified by RPE is bounded by the number of *h-bad* points (k^*). Adding the k^* bad points themselves (which are always misclassified) yields:

$$\mathbb{E}[\text{err}_P(h_{\text{RPE}})] \leq k^* + k^* = 2k^*.$$

Example 2.2. Let P be the set of points in Figure 2, and h be the classifier that maps all the black points to 1 except p_1 and all the white points to -1 except p_{11} and p_{15} . Thus, p_1 , p_{11} , and p_{15} are h -bad, while the other points of P are h -good. The following are some representative influence sets:

- $I_{\text{bad}}(p_{15}) = \{p_4, p_9, p_{10}, p_{12}, p_{13}, p_{14}, p_{16}\}$
- $I_{\text{bad}}(p_1) = \{p_2, p_3, p_5\}$
- $I_{\text{good}}(p_3) = \{p_1\}$
- $I_{\text{good}}(p_9) = \{p_{11}, p_{15}\}$

2. Cost Analysis

The efficiency of RPE is not measured by the total points n , but by the dominance width w (the size of the largest set of incomparable points). The proof is summarized as follows:

1. **Decomposition:** By Dilworth’s Theorem (Section 2.7.1), any point set P can be partitioned into w disjoint chains (where every point in a chain is comparable to the others).
2. **Parallel Search:** RPE effectively performs a randomized search on all w chains simultaneously. Each probe in P is guaranteed to land in one of these chains.
3. **A&E Game:** Within any single chain, RPE’s behavior mimics an “Attrition-and-Elimination” game (Section 2.7.2). Even if an adversary removes points to try and trick the algorithm, a random probe is likely to “hit” near the middle of the remaining chain, eliminating half of it.

By summing the expected probes across w chains and applying the concavity of the logarithm (Jensen’s Inequality), we arrive at the total expected cost: $O(w \log \frac{n}{w})$.

2.5 Another Worked Example: Why the Factor of 2 Is Tight

The error guarantee of $2k^*$ is the *best possible* approximation ratio for RPE. The following example demonstrates this:

Example 2.3. Consider Figure 3, where a point set P in $\mathbb{R}^2$ contains one black point (label $+1$) and $n - 1$ white points (label -1).

Optimal error: The monotone classifier that maps all points to -1 misclassifies only the black point, so $k^* = 1$.

RPE’s expected error: With probability $1/n$, RPE probes the black point first. Since it has label $+1$ and dominates nothing, no points are eliminated. So, all $n - 1$ white points remain and are later mapped to $+1$ by h_{RPE} , giving error $n - 1$. With probability $1 - 1/n$, RPE probes a white point first, eliminates all dominated points (just itself), and the resulting error is 1. Thus:

$$\mathbb{E}[\text{err}(h_{\text{RPE}})] = \frac{1}{n}(n - 1) + \left(1 - \frac{1}{n}\right)(1) = 2 - \frac{2}{n} \rightarrow 2 \quad \text{as } n \rightarrow \infty.$$

Since $k^* = 1$, the approximation ratio approaches $2k^*/k^* = 2$. The factor of 2 is therefore *asymptotically tight* for RPE.

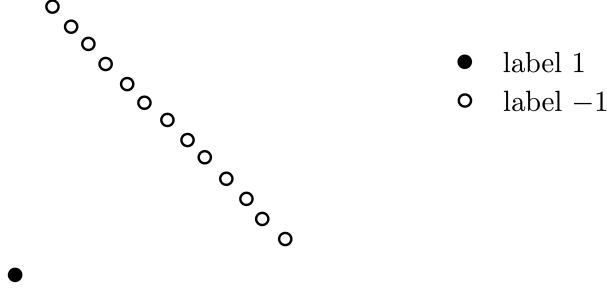


Figure 3: The approximation ratio 2 is tight for RPE.

2.6 Limitations

RPE has two key limitations:

1. **The factor-of-2 error is tight.** As shown in Example 2.3, no modification to RPE’s random-probing strategy can reduce the approximation ratio below 2. To achieve a ratio of $1 + \epsilon$ for arbitrary $\epsilon > 0$, a fundamentally different technique is needed. This is the motivation for the *relative-comparison coreset* approach in Section 3.
2. **No high-probability guarantee.** Theorem 2.1 gives an *expected* error bound. In a single run, RPE may produce an error much larger than $2k^*$, as the bad scenario in Example 2.3 illustrates. The coreset-based algorithm upgrades this to a *w.h.p.* guarantee at the cost of an extra $O(\log n / \epsilon^2)$ factor in probes.

Table 2: RPE vs. Coreset Algorithm

Property	RPE (Section 2)	Coreset Alg. (Section 3)
Error guarantee	$\mathbb{E}[\text{err}] \leq 2k^*$ (expected)	$(1 + \epsilon)k^*$ (w.h.p.)
Approx. ratio	2 (tight)	$1 + \epsilon$ (any $\epsilon > 0$)
Probing cost	$O(w \cdot \log(n/w))$	$O(\frac{w}{\epsilon^2} \cdot \log(n/w) \cdot \log n)$
Requires k^* ?	No	No
High-probability?	No	Yes

2.7 Chains, Dilworth’s Theorem, and the A&E Game (Supplementary)

This section covers supplementary material relevant to the cost analysis.

2.7.1 Chains and Dilworth’s Theorem

A *chain* is a subset $S \subseteq P$ where every two elements are comparable under dominance: we can order them as $p_1 \preceq p_2 \preceq \dots \preceq p_{|S|}$. A *chain decomposition* partitions P into disjoint chains. Dilworth’s theorem [?] guarantees the existence of a chain decomposition using exactly w chains (where w is the width of P).

Example 2.4. The input P in Figure 2 can be decomposed into 6 chains:

- $C_1 = \{p_1, p_2, p_3, p_4, p_{10}\}$
- $C_2 = \{p_{11}\}$
- $C_3 = \{p_5, p_9, p_{12}\}$

- $C_4 = \{p_{16}\}$
- $C_5 = \{p_{13}\}$
- $C_6 = \{p_6, p_7, p_8, p_{14}, p_{15}\}$

The points $p_{10}, p_{11}, p_{12}, p_{16}, p_{13}, p_{14}$ indicate the absence of any chain decomposition of P having less than 6 chains.

2.7.2 A&E Game

To analyze RPE's cost within a single chain, the paper introduces a two-player *Attrition-and-Elimination (A&E) game* on a chain C of m points. In each round:

- Bob (adversary) may delete any subset of points from C (attrition).
- Alice picks a uniformly random point $p \in C$ and deletes all points $q \succeq p$ (elimination).

The game ends when C is empty. This generalizes a **random binary search**: while a standard binary search assumes the points stay put, the A&E game proves that even if an adversary removes points to “hide” the middle, Alice will still finish quickly.

Lemma 2.3. Regardless of Bob's strategy, the expected number of rounds Y satisfies $\mathbb{E}[Y] = O(\log m)$.

The proof proceeds in two steps: showing that each round has a high probability of being “successful,” and then bounding the total rounds needed.

1. Probability of a “Successful” Round: Define a round as *successful* if it reduces the number of elements in the chain by at least a factor of 2.

- Let m' be the number of points left after Bob's attrition in a round.
- Let the points be ordered $p_1 \preceq p_2 \preceq \dots \preceq p_{m'}$.
- If Alice picks any point p_i where $i \leq \frac{m'}{2} + 1$, then at most half the points remain after she eliminates everything dominating p_i .
- Since Alice picks uniformly at random, the probability of picking such a point is $> 1/2$. Thus, every round is successful with probability $> 1/2$, regardless of what Bob does.

2. Total Expected Rounds: A chain of size m can only be halved $\log_2(m)$ times before it is empty. Therefore, we only need $\log_2(m)$ “successful” rounds to end the game.

- If we run $Y = 2\log_2(m)$ rounds, we expect half of them to be successful (which would be exactly $\log_2(m)$).
- Using the properties of independent trials, the probability that we fail to reach $\log_2(m)$ successes in $2\log_2(m)$ rounds is very small (less than $1/m$).
- Even in the worst-case scenario where we never get enough successes, the game must end in at most m rounds (since Alice removes at least one point per round).

Combining these, the expected number of rounds is dominated by the $2\log_2(m)$ term:

$$\mathbb{E}[Y] \leq 2\log_2(m) + m \cdot \mathbf{Pr}(\text{failure}) = O(\log m).$$

3 Relative-Comparison Coresets (RCC)

3.1 Why RPE Isn't Enough

Recall that RPE achieves an expected error of at most $2k^*$, and this guarantee is tight. In the entity-matching context a factor-2 margin may be unacceptable for a production pipeline.

The goal of this section is to achieve an approximation ratio of $(1 + \varepsilon)$ for any user-specified $\varepsilon > 0$: find a function $F : \mathbb{H}_{\text{mon}} \rightarrow \mathbb{R}$ such that

$$F(h) \leq F(h') \implies \text{err}_P(h) \leq (1 + \varepsilon) \text{err}_P(h').$$

Since $F(\hat{h}) \leq F(h^*)$ for any optimal h^* , this gives $\text{err}_P(\hat{h}) \leq (1 + \varepsilon)k^*$.

Remark 3.1 (Why estimating $\text{err}_P(h)$ directly fails). A natural idea is to set $F(h) = \text{err}_P(h)$ estimated by probing. But even for h^{pos} (the all-+1 classifier), estimating its error to within a constant relative factor requires finding the single -1 element (or confirming none exists), which demands $\Omega(n)$ probes in expectation.

3.2 The Unknown- Δ Technique

We sidestep the $\Omega(n)$ barrier by constructing F satisfying a weaker condition. Instead of estimating $\text{err}_P(h)$ exactly, we find an F where all estimates are shifted by the same unknown constant Δ :

$$\text{err}_P(h)(1 - \frac{\varepsilon}{4}) + \Delta \leq F(h) \leq \text{err}_P(h)(1 + \frac{\varepsilon}{4}) + \Delta \quad \forall h \in \mathbb{H}_{\text{mon}}, \quad (1)$$

Proposition 3.1. Any F satisfying (1) has the relative ε -comparison property.

Proof. Suppose $F(h) \leq F(h')$. Using the left inequality of (1) for h and the right for h' , then cancelling Δ :

$$\text{err}_P(h)(1 - \frac{\varepsilon}{4}) \leq F(h) - \Delta \leq F(h') - \Delta \leq \text{err}_P(h')(1 + \frac{\varepsilon}{4}).$$

Rearranging:

$$\text{err}_P(h) \leq \frac{1 + \varepsilon/4}{1 - \varepsilon/4} \text{err}_P(h') \leq (1 + \varepsilon) \text{err}_P(h'),$$

where the last step uses $\frac{1 + \varepsilon/4}{1 - \varepsilon/4} \leq 1 + \varepsilon$ for all $\varepsilon \in (0, 1]$. □

The key insight: Δ cancels whenever we *compare* two classifiers. We never need to know its value — only its existence.

3.2.1 Running Example

We will trace a small 1D instance through the entire construction. Let $P = \{1, 2, 3, 4, 5, 6, 7, 8\}$ with labels

$$\underbrace{-1, -1, -1, -1}_{\text{points 1-4}}, \underbrace{+1, +1, +1, +1}_{\text{points 5-8}}.$$

Every monotone 1D classifier has the form $h^\tau(p) = +1$ iff $p > \tau$. The optimal classifier is $h_{4.5}$, with $k^* = 0$. We set $\varepsilon = 1/2$.

3.3 The Coreset Technique

We realise F via a **relative-comparison coreset**: a weighted subset $Z \subseteq P$ whose weighted error $w\text{-err}_Z(h)$ satisfies (1) for all h .

Definition 3.1 (Relative-Comparison Coreset). A **relative-comparison coreset** of P is a subset $Z \subseteq P$ where every $z \in Z$ has its label revealed and carries a positive weight $\text{weight}(z)$, such that the *weighted error*

$$w\text{-err}_Z(h) := \sum_{z \in Z: h(z) \neq \text{label}(z)} \text{weight}(z)$$

satisfies (1) for every $h \in \mathbb{H}_{\text{mon}}$ — i.e., $w\text{-err}_Z$ serves as the desired function F .

The novelty of the unknown- Δ approach is that classical coresets approximate $\text{err}_P(h)$ itself (costing $\Omega(n)$ probes), whereas here Δ never needs to be computed.

3.4 Building the Coreset in 1D: The Recursive Framework

The 1D construction works by solving the following problem at every level of recursion. The coreset is built by the algorithm `BUILD CORESET`, which solves Problem 3.1 recursively.

Problem 3.1 (1D Function Approximation). Let P be a multiset of 1D labeled points (as in Problem 1.1 with $d = 1$), and $\varepsilon \in (0, 1]$. Find a function $F : \mathbb{H}_{\text{mon}} \rightarrow \mathbb{R}$ such that every $h \in \mathbb{H}_{\text{mon}}$ satisfies simultaneously:

$$|F(h) - \text{err}_P(h)| \leq \varepsilon |P| / 64, \quad (2)$$

$$\text{err}_P(h)(1 - \frac{\varepsilon}{4}) + \Delta \leq F(h) \leq \text{err}_P(h)(1 + \frac{\varepsilon}{4}) + \Delta, \quad (3)$$

for some (possibly unknown) Δ with $|\Delta| \leq \varepsilon |P| / 64$.

3.4.1 Estimating Error via Sampling (Functions G_1, G_2, F_α)

At each recursion level on a set $Q \subseteq P$, we draw a uniform sample S of size $O\left(\frac{1}{\varepsilon^2} \log \frac{|Q|\ell}{\delta}\right)$ from Q , where $\ell = O(\log n)$ is the number of recursion levels. Define

$$G(h) := \frac{|Q|}{|S|} \cdot \text{err}_S(h).$$

By a standard concentration bound, G satisfies condition (2) for Q simultaneously for all $h \in \mathbb{H}_{\text{mon}}$, with high probability. Three such estimates are used per level:

- G_1 , sampled from the full P , is used to *locate the split point*. Does not enter the coreset.
- G_2 , sampled from P_{rest} , approximates $\text{err}_{P_{\text{rest}}}(h)$ and contributes its sampled points to Z .
- F_α , sampled from P_α , approximates $\text{err}_{P_\alpha}(h)$ and contributes its sampled points to Z .

Together, the target function for Problem 3.1,

$$F(h) = G_2(h) + F_\alpha(h) + F_{\text{mid}}(h)$$

where F_{mid} comes from recursion on P_{mid} satisfies both conditions of Problem 3.1 for the full P .

As there are ℓ levels, the overall cost is $O\left(\frac{\ell}{\varepsilon^2} \log \frac{n}{\delta}\right) = O\left(\frac{\log n}{\varepsilon^2} \cdot \log \frac{n}{\delta}\right)$, which lets us solve Problem 3.1 with a probability of at least $1 - \delta$.

3.4.2 Example (continued)

- Suppose at the top level we draw $S_1 = \{2, 5, 7\}$ from P , with labels $-1, +1, +1$.
- For $h_{4.5}$ (optimal): $\text{err}_{S_1}(h_{4.5}) = 0$, so $G_1(h_{4.5}) = 0$. True error is also 0; condition (2) holds trivially ($|0 - 0| = 0 \leq 0.0625$).
- For $h_{2.5}$: $\text{err}_{S_1}(h_{2.5}) = 1$ (point 5 misclassified), so $G_1(h_{2.5}) = 8/3 \approx 2.67$. True error is 2; the gap is 0.67, well within the tolerance $\varepsilon|P|/64 = 0.0625 \cdot 8 = 0.5$... close to the boundary, illustrating why the sample size matters.

3.4.3 Splitting P into Three Parts

The purpose of G_1 — satisfying condition (2) for the full P — is to identify where the error function transitions through the value $|P|/4$. Concretely, define:

$$\alpha = \text{smallest } \tau \in P \cup \{-\infty\} \text{ with } G_1(h^\tau) < |P|\left(\frac{1}{4} - \frac{\varepsilon}{64}\right),$$

$$\beta = \text{largest } \tau \in P \cup \{-\infty\} \text{ with } G_1(h^\tau) < |P|\left(\frac{1}{4} - \frac{\varepsilon}{64}\right).$$

Because G_1 satisfies (2), any h^τ with $G_1(h^\tau)$ below this threshold has $\text{err}_P(h^\tau) < |P|/4$. This splits P into:

$$P_\alpha = \{p \in P : p = \alpha\}, \quad P_{\text{mid}} = \{p \in P : \alpha < p \leq \beta\}, \quad P_{\text{rest}} = P \setminus (P_\alpha \cup P_{\text{mid}}).$$

Crucially, classifiers h^τ with $\tau \notin [\alpha, \beta)$ have $\text{err}_P(h^\tau) \geq |P|/4$, so condition (3) for those classifiers follows from (2) alone (with $\Delta = 0$). The recursive step only needs to handle $\tau \in [\alpha, \beta)$, where errors are small and the unknown Δ is needed.

Proposition 3.2 (Size of P_{mid}). $|P_{\text{mid}}| < |P|/2$.

Proof. If $\alpha = \beta = \text{null}$, the claim is trivial. Otherwise, P_{mid} cannot contain $\geq |P|/4$ positive-label points (else $G_1(h_\beta) \geq |P|(1/4 - \varepsilon/64)$, contradicting the definition of β). By a symmetric argument, it cannot contain $\geq |P|/4$ negative-label points either. Hence $|P_{\text{mid}}| < |P|/2$. $\square$

Proposition 3.2 guarantees the recursion terminates in $O(\log n)$ levels.

3.4.4 Example (continued)

With $|P| = 8$ and $\varepsilon = 1/2$, the threshold is $8(1/4 - 1/128) \approx 1.94$. Suppose G_1 evaluates to roughly:

Classifier h^τ	True $\text{err}_P(h^\tau)$	$G_1(h^\tau)$ (approx.)
$h_{-\infty}$ (all +1)	4	4.0
h_1	3	3.2
h_2	2	2.1
h_3	1	1.2
h_4	0	0.1
h_5	1	0.8
h_6	2	2.2
h_7	3	3.1
h_8 (all -1)	4	4.0

G_1 drops below 1.94 at $\tau = 3$ and $\tau = 4$. So $\alpha = 3$, $\beta = 4$, giving:

$$P_\alpha = \{3\}, \quad P_{\text{mid}} = \{4\}, \quad P_{\text{rest}} = \{1, 2, 5, 6, 7, 8\}.$$

Note $|P_{\text{mid}}| = 1 < 4 = |P|/2$, consistent with Proposition 3.2.

3.4.5 Assembling the Coreset Z

The coreset is assembled level by level:

- A sample S_2 drawn from P_{rest} enters Z with weight $|P_{\text{rest}}|/|S_2|$.
- A sample S_α drawn from P_α enters Z with weight $|P_\alpha|/|S_\alpha|$.
- The recursion on P_{mid} returns a sub-coreset Z_{mid} , which is also included in Z .

Setting $F(h) = G_2(h) + F_\alpha(h) + F_{\text{mid}}(h)$ and $\Delta = \Delta_\alpha + \Delta_{\text{mid}} + (G_2(h_\beta) - \text{err}_{P_{\text{rest}}}(h_\beta))$, one can verify (see Section 3 of the paper) that F satisfies both conditions (2) and (3) of Problem 3.1 for the full P . Since $\text{w-err}_Z(h) = F(h)$, the coreset Z realises the desired function F .

Theorem 3.1 (Theorem 6 in the paper). Let n and w be the size and width of the input P , respectively. In

$$O\left(\frac{w}{\varepsilon^2} \log \frac{n}{w} \cdot \log \frac{n}{\delta}\right)$$

probes, we can obtain with probability at least $1 - \delta$ a relative-comparison coreset Z of P with

$$|Z| = O\left(\frac{w}{\varepsilon^2} \log \frac{n}{w} \cdot \log \frac{n}{\delta}\right).$$

3.4.6 Example (continued)

At the top level, S_2 is drawn from $P_{\text{rest}} = \{1, 2, 5, 6, 7, 8\}$ (size 6) and S_α from $P_\alpha = \{3\}$ (size 1). The recursion on $P_{\text{mid}} = \{4\}$ (size 1) trivially probes the single element. With weights assigned proportionally to the set sizes, w-err_Z correctly scales the sampled errors back to represent all of P .

3.5 Extension to Higher Dimensions

For $d > 1$, the algorithm first computes a **chain decomposition** $C_1, \dots, C_w$ of P using Dilworth's Theorem — the same decomposition used in the cost analysis of RPE (Section 2). Since each chain C_i is ordered, it can be treated as a 1D instance.

Applying the 1D coreset construction to each C_i independently yields sub-coresets $Z_1, \dots, Z_w$. The final coreset is $Z = \bigcup_{i=1}^w Z_i$. For every $h \in \mathbb{H}_{\text{mon}}$:

$$\text{err}_P(h) = \sum_{i=1}^w \text{err}_{C_i}(h), \quad \text{w-err}_Z(h) = \sum_{i=1}^w \text{w-err}_{Z_i}(h).$$

Since each Z_i satisfies a per-chain relative bound with unknown shift Δ_i , summing gives the global bound with $\Delta = \sum_i \Delta_i$.

3.6 Achieving the $(1 + \varepsilon)$ Guarantee

Given the coreset Z , finding the best monotone classifier requires *no further probing*: simply compute

$$\hat{h} = \arg \min_{h \in \mathbb{H}_{\text{mon}}} \text{w-err}_Z(h),$$

which can be done in time polynomial in $|Z|$ and d .

Corollary 3.1 (Corollary 7 in the paper). For Problem 1 with any $\varepsilon \in (0, 1]$, there exists an algorithm that finds, with high probability, a monotone classifier with error at most $(1 + \varepsilon)k^*$ using

$$O\left(\frac{w}{\varepsilon^2} \log \frac{n}{w} \cdot \log n\right)$$

probes, where $n = |P|$, w is the width of P , and k^* is the optimal monotone error.

The argument is direct: for any $h \in \mathbb{H}_{\text{mon}}$, the coreset bound gives

$$\text{err}_P(\hat{h})(1 - \frac{\varepsilon}{4}) \leq \text{w-err}_Z(\hat{h}) - \Delta \leq \text{w-err}_Z(h^*) - \Delta \leq \text{err}_P(h^*)(1 + \frac{\varepsilon}{4}),$$

so setting $h = h^*$ (the true optimal) and rearranging yields $\text{err}_P(\hat{h}) \leq (1 + \varepsilon)k^*$.

3.6.1 Example (conclusion)

We compute $\text{w-err}_Z(h^\tau)$ for each threshold τ . The minimiser is $h_{4.5}$ (equivalently h_4), with weighted error ≈ 0 . Since $k^* = 0$, the $(1 + \varepsilon)k^* = 0$ bound is met exactly.

4 Full Algorithmic Description

Algorithm 1 BuildCoreset(P)

```

1:  $Z \leftarrow \emptyset$ 
2: if  $|P| = 1$  then
3:   Probe the (only) element  $p \in P$ 
4:    $Z \leftarrow \{p\}$  with  $\text{weight}(p) = 1$ 
5: else
6:   Probe sample  $S_1$  of size  $O\left(\frac{1}{\varepsilon^2} \log\left(\frac{|P|\ell}{\delta}\right)\right)$  from  $P$ 
7:   {Defines  $G_1(h) = (|P|/|S_1|) \cdot \text{err}_{S_1}(h)$ ; use  $G_1$  to find  $\alpha, \beta$  and split  $P$  into  $P_\alpha, P_{\text{mid}}, P_{\text{rest}}$ }
8:   if  $P_{\text{rest}} \neq \emptyset$  then
9:     Probe sample  $S_2$  of size  $O\left(\frac{1}{\varepsilon^2} \log\left(\frac{|P|\ell}{\delta}\right)\right)$  from  $P_{\text{rest}}$ 
10:    {Defines  $G_2(h) = (|P_{\text{rest}}|/|S_2|) \cdot \text{err}_{S_2}(h)$ }
11:    Add  $S_2$  to  $Z$  with  $\text{weight}(p) = |P_{\text{rest}}|/|S_2|$  for each  $p \in S_2$ 
12:   end if
13:   if  $P_\alpha \neq \emptyset$  then
14:     Probe sample  $S_\alpha$  of size  $O\left(\frac{1}{\varepsilon^2} \log\left(\frac{|P|\ell}{\delta}\right)\right)$  from  $P_\alpha$ 
15:     {Defines  $F_\alpha(h) = (|P_\alpha|/|S_\alpha|) \cdot \text{err}_{S_\alpha}(h)$ }
16:     Add  $S_\alpha$  to  $Z$  with  $\text{weight}(p) = |P_\alpha|/|S_\alpha|$  for each  $p \in S_\alpha$ 
17:   end if
18:    $Z_{\text{mid}} \leftarrow \text{BUILD CORESET}(P_{\text{mid}})$ 
19:   Add  $Z_{\text{mid}}$  to  $Z$ 
20: end if
21: return  $Z$ 

```

5 Proofs and Lower Bounds

5.1 Lower Bound Results

5.2 Near-Optimality of the Algorithms

5.3 Mathematical Machinery

Summary

References

- [1] Y. Tao, “Monotone Classification,” *PODS’18*.
- [2] Y. Tao, “Monotone Classification (Extended),” *PODS’21*.